

UNIVERSIDAD MARIANO GALVEZ DE GUATEMALA



CATEDRATICO: Marco Tulio

ALUMNA: Enma Leticia Ramírez Castro

CARNET: 0907-23-18206

CURSO: Análisis de Sistemas

CARRERA: Ingeniería en Sistemas

TAREA: Investigación de patrones de diseño

CICLO: Séptimo

SECCION: "A"

INTRODUCCIÓN

Los patrones de diseño son soluciones reutilizables a problemas comunes que aparecen durante el desarrollo de software. Estos patrones ayudan a crear aplicaciones más organizadas, mantenibles y escalables. Además, permiten que diferentes desarrolladores puedan entender mejor la estructura de un proyecto y facilitan la reutilización de código.

En esta investigación se describen algunos de los patrones de diseño más utilizados, incluyendo su propósito, funcionamiento y ejemplos prácticos en programación.

1) Factory Method

Es un patrón creacional que permite crear objetos sin especificar directamente la clase exacta que se utilizara. En lugar de crear objetos utilizando el operador new en varias partes del programa, se delega la creación a un método especializado.

¿Que aprendí?

Comprendí que este patrón sirve para centralizar la creación de objetos y evitar depender directamente de clases concretas. Esto hace que el código se más flexible y fácil de modificar en el futuro.

Ventajas:

- Reduce el acoplamiento
- Facilita agregar nuevos tipos de objetos
- Mejorar la organización del código

2) Abstract Builder

Permite crear objetos completos paso a paso utilizando una interfaz abstracta. Se utiliza cuando un objeto tiene muchas configuraciones posibles.

¿Que aprendí?

Comprendí que este patrón ayuda a separar la construcción de un objeto de su representación final. De esta manera, el mismo proceso puede generar diferentes resultados dependiendo de la implementación.

Ventajas:

- ✓ Facilita la creación de objetos complejos.
- ✓ Permite reutilizar el mismo proceso de construcción
- ✓ Mejora la legibilidad del código

3) Builder

Permite construir objetos complejos paso a paso. A diferencia de crear el objeto en un solo constructor, el Builder divide la creación en diferentes etapas.

¿Que aprendí?

Este patrón es útil cuando una clase tiene muchos atributos y varios parámetros opcionales. Ayuda a evitar construcciones demasiado largos y difíciles de entender.

Ventajas:

- Código más limpio
- Facilita la creación de objetos configurables.
- Evita constructores complicados.

4) Prototype

El patrón Prototype permite crear nuevos objetos copiando otros objetos existentes.

¿Que aprendí?

Este patrón es útil cuando crear un objeto desde cero es costoso o complicado. En lugar de crear uno nuevo, se realiza una copia del objeto original.

Ventajas:

- Reduce tiempo de creación
- Permite duplicar configuraciones existentes.
- Simplifica la creación de objetos complejos

5) Singleton

El patrón Singleton garantiza que una clase tenga una única instancia y proporciona un punto de acceso global a ella.

¿Que aprendí?

Este patrón se utiliza cuando solo debe existir un objeto en todo el sistema, como una conexión a base de datos o un administrador de configuración.

Ventajas:

- Evita multiples instancias innecesarias
- Controla el acceso global
- Ahorra recursos

6) Adapter

El patrón Adapter permite que dos clases incompatibles pueden trabajar juntas.

¿Que aprendí?

Comprendi que este patrón funciona como un adaptador o traductor entre dos estructuras diferentes para que puedan comunicarse.

Ventajas:

- Reutiliza código existente
- Facilita integración entre sistemas
- Reduce cambios en código viejo

7) Bridge

Este patrón Bridge separa una abstracción de su implementación para que ambas puedan modificarse independientemente.

¿Que aprendí?

Comprendi que este patrón ayuda a dividir funcionalidades grandes en partes más pequeñas y felxibles.

Ventajas:

- ❖ Reduce dependencias
- ❖ Facilita cambios independientes
- ❖ Mejora la estabilidad

8) Composite

Permite tratar objetos individuales y grupos de objetos de la misma manera

¿Que aprendí?

Este patrón sirve para organizar estructuras jerárquicas, como carpetas y archivos, donde un grupo puede contener otros elementos.

Ventajas:

- Facilita manejo de estructuras complejas
- Permite trabajar de manera uniforme
- Mejora la organización jerarquica

CONCLUSIÓN.

Los patrones de diseño son soluciones reutilizables que permiten resolver problemas comunes en el desarrollo de software de manera estructurada. A través de esta investigación, se comprendió que cada patrón tiene un propósito específico, ya sea para la creación de objetos, la organización de estructuras o la comunicación entre clases.

La implementación en Kotlin permitió reforzar el entendimiento práctico de estos conceptos observando cada patrón mejora la optimización, estabilidad y mantenimiento del código.

En conclusión, el uso de patrones de diseño es fundamental en el desarrollo de software moderno, ya que facilita la creación de sistemas más flexibles, reutilizables y fáciles de manejar.

Link de GitHub:

<https://github.com/EnmaRamirez/Patrones-de-dise-o.git>